

## Вопрос 1

- **Вопрос по Java:** Понятия объекта и инкапсуляции. Понятие класса. Объявление классов в Java. Члены класса. Доступ к членам класса.
- **Вопрос по C++:** Понятие шаблона. Типовые формальные параметры шаблона. Нетиповые формальные параметры шаблона. Шаблонные формальные параметры шаблона.

### Ответ по Java:

#### 1. Понятия объекта и инкапсуляции (Слайд 9)

- **Объект (object)** – это самоописывающая структура данных, обладающая внутренним состоянием и способная обрабатывать передаваемые ей сообщения. В Java набор сообщений, которые может обрабатывать объект, фиксирован на этапе компиляции.
- **Инкапсуляция (incapsulation)** – один из основных принципов ООП, заключающийся в том, что доступ к внутреннему состоянию объекта извне осуществляется только через механизм передачи сообщений.

#### 2. Понятие класса (Слайд 10)

- **Класс (class)** – это тип данных, значениями которого являются объекты, имеющие сходное внутреннее состояние и обрабатывающие одинаковый набор сообщений. Класс можно рассматривать как шаблон для порождения объектов (экземпляров класса).

#### 3. Объявление классов в Java (Слайд 11)

- Классы делятся на **публичные** и **непубличные**. В каждом `.java`-файле может быть только один публичный класс, и имя файла должно совпадать с именем этого класса.
- Синтаксис: `public class Имя { ... }`. Непубличные классы объявляются без `public`. Точка с запятой после объявления не ставится.

#### 4. Члены класса (Слайд 12) К членам класса относятся: экземплярные и статические поля, экземплярные и статические методы, экземплярные и статические конструкторы, вложенные классы.

#### 5. Доступ к членам класса (Слайд 13)

- Доступ осуществляется через операцию «`.`».
- Управляется **модификаторами доступа**: `private` (доступ только из тела класса), **без модификатора** (доступ в пределах пакета), `protected` (доступ в пакете и у наследников), `public` (доступ отовсюду).

### Ответ по C++:

#### 1. Понятие шаблона (Слайд 50)

- В C++ используется развитый язык макроопределений для порождения кода во время компиляции. Ключевым элементом является **шаблон**, который является "рецептом" для генерации кода класса или функции.
- Объявление начинается с ключевого слова `template`, за которым следует список формальных параметров: `template <формальные параметры> определение_класса_или_функции`.

#### 2. Типовые формальные параметры шаблона (Слайд 51)

- Обозначают типы. В списке параметров имя такого параметра предваряется ключевым словом `typename` (или `class`).
- Пример: `template <typename T> void do_swap(T& a, T& b);`

### 3. Нетиповые формальные параметры шаблона (Слайд 52)

- Представляют значения, а не типы. Их синтаксис похож на объявление переменных. Внутри шаблона имя такого параметра обозначает константу.
- Пример: `template <typename T, int M, int N> class Matrix;`

### 4. Шабононные формальные параметры шаблона (Слайд 53)

- Позволяют параметризовать один шаблон другим шаблоном. Синтаксически это выглядит как объявление шаблона без тела.
  - Пример: `template <template <typename, int, int> class M, ...> void multiply(...);`
- 

## Вопрос 2

- **Вопрос по Java:** Экземплярные поля. Статические поля.
- **Вопрос по C++:** Виртуальное наследование.

### Ответ по Java:

#### 1. Экземплярные поля (Слайд 14)

- **Определение:** Именованная составная часть внутреннего состояния объекта.
- **Назначение:** Хранение состояния, уникального для каждого экземпляра класса.
- **Объявление:** `public String name;`

#### 2. Статические поля (Слайд 15)

- **Определение:** Поле, разделяемое всеми объектами одного класса.
- **Назначение:** Хранение общих (разделяемых) данных для всех экземпляров.
- **Объявление:** Используется модификатор `static`. `public static int count;`

### Ответ по C++:

#### 1. Проблема, решаемая виртуальным наследованием (Слайды 46-47)

- При множественном наследовании может возникнуть "ромбовидная" (противоречивая) иерархия, когда класс-наследник получает один и тот же базовый класс по нескольким путям.
- Основная проблема такой иерархии — **многократное включение полей базового класса** в производный класс. Например, если `D` наследует `B` и `C`, а `B` и `C` наследуют `A`, то объект класса `D` будет содержать поля из `A` дважды.

#### 2. Виртуальное наследование (Слайд 48)

- Это способ реализации наследования, который **гарантирует, что базовый класс не будет включён в производный класс более чем в одном экземпляре**, даже если он встречается в иерархии несколько раз.
  - **Синтаксис:** При наследовании используется ключевое слово `virtual`. `class B: public virtual A { ... }; class C: public virtual A { ... }; class D: public B, public C { ... }; // Поля из A будут включены только один раз`
- 

## Вопрос 3

- **Вопрос по Java:** Экземплярные и статические методы. Объявление методов в Java. Перегрузка методов. Виртуальные методы.

- **Вопрос по C++:** Иерархия наследования и классы противоречия. Основная проблема противоречивых иерархий.

#### Ответ по Java:

1. **Экземплярные и статические методы (Слайд 16)**
  - **Экземплярный метод:** обрабатывает сообщение, имеет доступ к состоянию объекта через `this`.
  - **Статический метод:** не имеет доступа к состоянию конкретного объекта.
2. **Объявление методов в Java (Слайд 17)**
  - Объявляются внутри класса. Статические — с модификатором `static`.
3. **Перегрузка методов (Слайд 18)**
  - Объявление в классе методов с одинаковым именем, но **разными сигнатурами** (количество и/или типы параметров). Выбор метода происходит на этапе компиляции.
4. **Виртуальные методы (Слайд 20)**
  - Методы, для которых используется **позднее связывание** (адрес метода определяется в runtime по реальному типу объекта). **В Java все методы виртуальные.**

#### Ответ по C++:

1. **Иерархия наследования и классы противоречия (Слайд 46)**
  - **Иерархия наследования** – это ориентированный ациклический граф, узлами которого являются классы.
  - Класс **B** называется **классом противоречия**, если существует такой класс **A**, что в иерархии можно провести не менее двух непересекающихся путей из **A** в **B**. Иерархия с такими классами называется **противоречивой**.
2. **Основная проблема противоречивых иерархий (Слайд 47)**
  - Основной проблемой является **возможность многократного включения полей базового класса** в производный класс.
  - **Пример:**

```
class A { private: int x; };  
class B: public A {};  
class C: public A {};  
class D: public B, public C {};
```

- В этом примере поле `x` будет содержаться в объекте класса **D** дважды.

---

#### Вопрос 4

- **Вопрос по Java:** Понятие экземплярного конструктора. Конструктор по умолчанию. Объявление конструктора в Java.
- **Вопрос по C++:** Наследование от нескольких базовых классов. Противоречия в именах наследуемых членов класса. Разрешение противоречий в именах.

#### Ответ по Java:

### 1. Понятие экземплярного конструктора (Слайд 21)

- Это метод, предназначенный для **инициализации** только что созданного объекта. Его вызов объединен с созданием объекта, что гарантирует отсутствие неинициализированных объектов.

### 2. Конструктор по умолчанию (Слайд 22)

- Это конструктор с пустой сигнатурой (без параметров). Создается компилятором автоматически, если в классе не определено ни одного конструктора.

### 3. Объявление конструктора в Java (Слайд 23)

- Это метод, имя которого совпадает с именем класса и у которого отсутствует возвращаемое значение. Может быть перегружен. `public Point(double x, double y) { ... }`

## Ответ по C++:

### 1. Наследование от нескольких базовых классов (Слайд 43)

- C++ поддерживает множественное наследование.
- **Синтаксис:** Базовые классы перечисляются через запятую после двоеточия. `class Y: public X1, public X2, ..., public Xn { ... };`
- В конструкторе производного класса должны быть вызваны конструкторы всех базовых классов (кроме тех, у кого есть конструктор по умолчанию).

### 2. Противоречия в именах наследуемых членов (Слайд 44)

- При множественном наследовании может возникнуть **неоднозначность (ambiguity)**, если в производный класс из разных базовых классов приходят члены (методы или поля) с одинаковыми именами и сигнатурами.
- Попытка обратиться к такому члену без уточнения приведет к ошибке компиляции. `c.m();`  
`// error: request for member 'm' is ambiguous`

### 3. Разрешение противоречий в именах (Слайд 45)

- Для разрешения неоднозначности необходимо использовать **квалифицированные имена**, явно указывая, из какого базового класса вызывается член. `c.A::m();` // ОК, вызывается метод m из класса A

---

## Вопрос 5

- **Вопрос по Java:** Размещение объектов в памяти. Операция «new». Массивовые литералы.
- **Вопрос по C++:** Динамическое приведение типов.

## Ответ по Java:

### 1. Размещение объектов в памяти (Слайд 24)

- Объекты в Java располагаются только в динамической памяти (куче). Переменные, параметры и поля хранят не сами объекты, а **объектные ссылки** на них.

### 2. Операция «new» (Слайд 25)

- Создает объект в куче.
- Для объектов: `new ИмяКласса(параметры_конструктора);`
- Для массивов: `new тип_элемента[размер];`

### 3. Массивовые литералы (Слайд 26)

- Специальная форма `new` для создания и инициализации массива. `new int[] { 1, 2, 3 };`

#### Ответ по C++:

##### 1. Операция `dynamic_cast` (Слайд 39)

- Выполняет динамическое (во время выполнения) приведение типа объекта.
- **Синтаксис:** `dynamic_cast<T>(obj)`
- **Поведение:**
  - Проверяет, является ли тип `T` одним из типов объекта `obj`.
  - При работе с **указателями**: если приведение успешно, возвращает указатель на объект, иначе возвращает **нулевой указатель**.
  - При работе со **ссылками**: если приведение успешно, возвращает ссылку, иначе порождает исключение `std::bad_cast`.

##### 2. Ограничение на использование (Слайд 40)

- `dynamic_cast` можно применять только к **полиморфным типам**, то есть к классам, в которых есть хотя бы один **виртуальный метод**. Если тип не полиморфный, возникнет ошибка компиляции. Это связано с тем, что информация о типе, необходимая для приведения, хранится в объекте только для полиморфных классов.

---

#### Вопрос 6

- **Вопрос по Java:** Понятие статического конструктора. «Static»-блоки.
- **Вопрос по C++:** Основной способ наследования в C++. Переопределение методов.

#### Ответ по Java:

##### 1. Понятие статического конструктора (Слайд 27)

- Это статический метод для инициализации статических полей класса. Вызывается неявно при первом обращении к классу.

##### 2. «Static»-блоки (Слайд 28)

- В Java роль статического конструктора выполняет совокупность «**static**»-блоков.
- Синтаксис: `static { ... }`
- Блоки выполняются в порядке их объявления в классе при первой загрузке класса.

#### Ответ по C++:

##### 1. Основной способ наследования в C++ (Слайд 35)

- Синтаксис одиночного наследования: `class ИмяПроизводного: public ИмяБазового { ... };`
- В конструкторе производного класса необходимо явно вызывать конструктор базового класса (если у него нет конструктора по умолчанию) через **список инициализации**.  
`Dog::Dog(...) : Animal(...) { ... }`

##### 2. Переопределение методов (Слайды 37-38)

- Для переопределения метода, унаследованного от базового класса, достаточно объявить и определить этот метод в производном классе.
- Чтобы переопределение работало полиморфно (через указатель на базовый класс), метод в базовом классе должен быть объявлен как **virtual**.

```
// В базовом классе
virtual void draw();
// В производном классе
virtual void draw(); // Переопределяет метод базового класса
```

---

## Вопрос 7

- **Вопрос по Java:** Понятие субтипизации. Явная и неявная субтипизация.
- **Вопрос по C++:** Проблема копирования объектов. Объявление конструктора копий. Перегруженная операция присваивания.

### Ответ по Java:

#### 1. Понятие субтипизации (Слайд 30)

- Тип А является **подтипом** В, если код, написанный для В, может корректно работать с А.
- **Субтипизация** — это свойство языка, позволяющее использовать подтипы в программах.

#### 2. Явная и неявная субтипизация (Слайд 31)

- **Неявная (структурная):** решение о совместимости типов принимается на основе их структуры.
- **Явная (номинальная):** тип А является подтипом В, только если это явно объявлено (через наследование). **В Java используется явная субтипизация.**

### Ответ по C++:

#### 1. Проблема копирования объектов (Слайд 32)

- По умолчанию C++ выполняет **побитовое копирование** внутреннего состояния объекта.
- Это становится проблемой для классов, содержащих указатели на динамически выделенную память. В результате копирования два объекта (оригинал и копия) будут указывать на одну и ту же область памяти. Уничтожение одного объекта приведет к освобождению этой памяти, и указатель во втором объекте станет "висячим".

#### 2. Объявление конструктора копий (Слайд 33)

- Для создания "правильных" (глубоких) копий необходимо объявить **конструктор копий**. Он вызывается при инициализации объекта другим объектом.
- **Прототип:** `Имя_класса(const Имя_класса& obj);`
- В его теле нужно выделить новую память и скопировать в нее данные из исходного объекта.

#### 3. Перегруженная операция присваивания (Слайд 34)

- Проблема копирования возникает и при присваивании. Для ее решения перегружается оператор `=`.
- **Прототип:** `Имя_класса& operator=(const Имя_класса& obj);`
- Реализация должна включать **проверку на самоприсваивание** (`if (this != &obj)`), освобождение старой памяти, выделение новой и копирование данных.

---

## Вопрос 8

- **Вопрос по Java:** Явная и неявная субтипизация. Вызов конструктора базового класса. Операция приведения типа. Переопределение метода.
- **Вопрос по C++:** Объекты в глобальной памяти. Объекты в полях других объектов.

#### Ответ по Java:

1. **Явная и неявная субтипизация (Слайд 31):** В Java используется **явная субтипизация**, основанная на наследовании.
2. **Вызов конструктора базового класса (Слайд 34):** Осуществляется из конструктора производного класса с помощью `super(параметры);` и должен быть первой инструкцией.
3. **Операция приведения типа (Слайд 35):** Динамическое приведение (**Тип**)**объект**. В случае неудачи порождает `ClassCastException`.
4. **Переопределение метода (Слайд 36):** Замена реализации унаследованного метода в производном классе. В Java все методы виртуальные и могут быть переопределены.

#### Ответ по C++:

1. **Объекты в глобальной памяти (Слайд 29)**
  - Если объект размещается в глобальной переменной (или статическом поле класса), его конструктор вызывается **до** передачи управления в функцию `main`.
  - Деструктор для такого объекта вызывается **после** завершения функции `main`.

```
Demo d(100); // Глобальный объект
int main() { /* ... */ }
// Вывод: cons:100 main destr:100
```

2. **Объекты в полях других объектов (Слайды 30-31)**
  - Если объект класса **X** является полем **x** объекта класса **Y**, то конструктор для поля **x** вызывается из конструктора **Y** через **список инициализации**. `Y::Y(...) : x(...) { ... }`
  - Деструктор для объекта-поля (**d** в классе `Cont`) вызывается **автоматически** при уничтожении объекта-контейнера (**c**).

---

## Вопрос 9

- **Вопрос по Java:** Абстрактные классы.
- **Вопрос по C++:** Объявление деструктора. Объекты в автоматической памяти. Автоматический вызов деструктора.

#### Ответ по Java:

1. **Абстрактный метод (Слайд 38):** Виртуальный метод без тела, объявленный с модификатором `abstract`. Должен быть переопределен в производных классах.
2. **Абстрактный класс (Слайд 38):** Класс, содержащий абстрактные методы. Объявляется с модификатором `abstract`. **Создавать экземпляры абстрактных классов запрещено.**

#### Ответ по C++:

### 1. Объявление деструктора (Слайд 24)

- **Деструктор** – это экземплярный метод, предназначенный для освобождения ресурсов, принадлежащих объекту, непосредственно перед его уничтожением.
- В C++ без сборки мусора деструкторы жизненно необходимы.
- **Прототип:** `~имя_класса();`
- Деструктор вызывается при уничтожении объекта, в частности, его автоматически вызывает оператор `delete`.

### 2. Объекты в автоматической памяти (Слайд 26)

- В C++, в отличие от Java, объекты могут размещаться в автоматической памяти (на стеке) как локальные переменные.
- Синтаксис: `Имя_класса имя_переменной(параметры_конструктора);`

### 3. Автоматический вызов деструктора (Слайд 26, 28)

- При выходе из блока (например, из функции), где была объявлена переменная с объектом в автоматической памяти, для этого объекта **автоматически вызывается деструктор**. Это обеспечивает детерминированное управление ресурсами (принцип RAII).

---

## Вопрос 10

- **Вопрос по Java:** Объявление интерфейсов. Реализация интерфейсов. Наследование интерфейсов.
- **Вопрос по C++:** Создание объектов в динамической памяти. Создание массивов в динамической памяти. Удаление объектов и массивов.

### Ответ по Java:

1. **Объявление интерфейсов (Слайд 40):** Набор абстрактных методов, объявляется с помощью `interface`. `interface MovableShape { ... }`
2. **Реализация интерфейсов (Слайд 42):** Класс реализует интерфейс с помощью `implements`. Класс должен определить все методы интерфейса. Класс может реализовывать несколько интерфейсов.
3. **Наследование интерфейсов (Слайд 43):** Интерфейс может наследовать другие интерфейсы с помощью `extends`. Для интерфейсов реализовано множественное наследование.

### Ответ по C++:

1. **Создание объектов в динамической памяти (Слайд 21)**
  - Динамическое выделение памяти под объект выполняется операцией `new`. Она совмещает выделение памяти и вызов конструктора.
  - **Синтаксис:** `new имя_класса(фактические_параметры_конструктора);`
  - Операция возвращает указатель на созданный объект. `Point *p = new Point(1.5, 2.3);`
2. **Создание массивов в динамической памяти (Слайд 22)**
  - Для создания массивов используется специальная форма `new`.
  - **Синтаксис:** `new тип_элемента[размер];`
  - **Важно:** для создания массива объектов в динамической памяти у класса должен быть определён **конструктор по умолчанию**, который будет вызван для каждого элемента массива.

### 3. Удаление объектов и массивов (Слайд 23)

- Так как в C++ нет сборщика мусора, все объекты, созданные через `new`, должны быть явно освобождены оператором `delete`.
  - **Для одиночного объекта:** `delete указатель_на_объект;`
  - **Для массива:** `delete [] указатель_на_массив;` (использование `[]` обязательно для корректного вызова деструкторов для всех элементов).
- 

## Вопрос 11

- **Вопрос по Java:** Экземплярные и статические вложенные классы. Локальные классы. Анонимные классы.
- **Вопрос по C++:** Объявление конструкторов.

### Ответ по Java:

1. **Вложенные классы (Слайд 44):**
  - **Экземплярный (inner class):** имеет доступ к состоянию объекта объемлющего класса.
  - **Статический:** имеет доступ только к статическим членам объемлющего класса, объявляется с `static`.
2. **Локальные классы (Слайд 46):** Экземплярный класс, объявленный внутри метода. Имеет доступ к `final` переменным и параметрам метода.
3. **Анонимные классы (Слайд 47):** Локальный класс без имени, объявление которого совмещено с созданием экземпляра. `new ИмяИнтерфейса() { ... };`

### Ответ по C++:

1. **Объявление конструкторов (Слайд 19)**
    - Объявление конструктора состоит из **прототипа** в теле класса и **определения** в `src`-файле.
    - Имя конструктора **совпадает с именем класса**.
    - Тип возвращаемого значения **не указывается**.
    - Чтобы запретить создание экземпляров, конструктор можно объявить в `private` или `protected` секции.
  2. **Особенности (Слайды 19-20)**
    - В C++ **никогда не следует вызывать виртуальные методы из конструктора**, так как это может привести к проблемам при наследовании и переопределении.
    - Для доступа к полям объекта из конструктора используется указатель `this`. `this->x = x;`
- 

## Вопрос 12

- **Вопрос по Java:** Функциональные интерфейсы и лямбда-выражения.
- **Вопрос по C++:** Объявление полей. Определения статических полей. Объявление методов. Виртуальные и абстрактные методы.

### Ответ по Java:

1. **Функциональный интерфейс (Слайд 48):** Интерфейс с единственным абстрактным методом.

2. **Лямбда-выражения (Слайд 48):** Синтаксический сахар для создания экземпляра анонимного класса, реализующего функциональный интерфейс.
  - **Формы:** (параметры) -> тело\_метода или (параметры) -> выражение.
  - Пример: (x, y) -> x + y;

Ответ по C++:

1. **Объявление полей (Слайд 15)**
    - Экземплярные поля объявляются так же, как поля структур в C.
    - Статические поля объявляются с модификатором `static`.
  2. **Определения статических полей (Слайд 16)**
    - Каждое статическое поле должно быть **определено** (инициализировано) в одном из src-файлов проекта, так как оно является, по сути, глобальной переменной.
    - Синтаксис: `тип Имя_класса::имя_поля = значение;`
  3. **Объявление методов (Слайд 17)**
    - Объявление состоит из **прототипа** в теле класса и **определения** в src-файле.
    - Прототипы статических методов объявляются с `static`.
  4. **Виртуальные и абстрактные методы (Слайд 18)**
    - **Виртуальный метод:** для включения позднего связывания прототип метода помечается модификатором `virtual`. `virtual double dist();`
    - **Абстрактный (чисто виртуальный) метод:** прототип такого метода заканчивается на `= 0;`. Класс, содержащий хотя бы один абстрактный метод, автоматически становится абстрактным. `virtual double dist() = 0;`
- 

## Вопрос 13

- **Вопрос по Java:** Необходимость обобщений. Контейнерные классы. Основная проблема при использовании необобщённых контейнерных классов.
- **Вопрос по C++:** Объявление класса. Секции в объявлении класса.

Ответ по Java:

1. **Контейнерные классы (Слайд 50):** Классы-хранилища для других объектов (`ArrayList`, `HashMap`).
2. **Необходимость обобщений и проблема (Слайд 53):**
  - Необобщённые контейнеры работают с типом `Object`.
  - **Основная проблема:** отсутствие безопасности типов на этапе компиляции. Можно по ошибке положить в контейнер объект не того типа, и компилятор это пропустит. Ошибка (`ClassCastException`) проявится только во время выполнения.

Ответ по C++:

1. **Объявление класса (Слайд 12)**
  - Классы объявляются с помощью конструкции `class Имя { ... };`.
  - В отличие от `struct`, где члены по умолчанию `public`, в `class` члены по умолчанию `private` (недоступны извне).
  - В теле класса располагаются объявления полей и прототипы методов.
2. **Секции в объявлении класса (Слайд 13)**

- Для управления доступом к членам класса используются секции-спецификаторы:
    - **public**:: члены видны отовсюду.
    - **private**:: члены видны только из методов данного класса.
    - **protected**:: члены видны из методов данного класса и его классов-наследников.
  - Секции могут идти в любом порядке и количестве.
- 

## Вопрос 14

- **Вопрос по Java:** Понятие обобщённого класса. Ограниченные обобщённые классы.
- **Вопрос по C++:** Ссылки как возвращаемые значения функций. Константные ссылки.

### Ответ по Java:

1. **Понятие обобщённого класса (Слайд 54):** Класс, имеющий формальные типовые параметры (`<T>`). При использовании класса на их место подставляются фактические типы, что обеспечивает безопасность типов. `class Stack<T> { ... }`
2. **Ограниченные обобщённые классы (Слайд 57):** Класс, где для типового параметра задана **верхняя граница** с помощью `extends`. Это позволяет использовать в качестве фактического типа только указанный класс или его наследников. `class Stack<T extends Number> { ... }`

### Ответ по C++:

1. **Ссылки как возвращаемые значения функций (Слайд 10)**
  - Функция может возвращать ссылку. Это позволяет использовать результат вызова функции в левой части операции присваивания, то есть изменять значение, на которое указывает возвращенная ссылка.

```
char& ith(char *s, int i) { return s[i]; }  
// ...  
ith(s, 2) = 'x'; // Изменяем 2-й символ в строке s
```

- **Важно:** нельзя возвращать ссылку на локальную переменную, так как она будет уничтожена после выхода из функции.
2. **Константные ссылки (Слайд 11)**
    - Объявляются с модификатором `const`.
    - **Синтаксис:** `const тип& имя_переменной;`
    - Константная ссылка позволяет читать значение, на которое она указывает, но **запрещает его изменять**. Попытка изменить значение через константную ссылку приведет к ошибке компиляции.

```
const int& y = x;  
y = 20; // Ошибка!
```

## Вопрос 15

- **Вопрос по Java:** Ковариантность массивов. Инвариантность обобщённых классов. Шаблоны обобщённых классов. Особенности шаблонов.
- **Вопрос по C++:** Объявление переменных ссылочного типа. Инициализация и использование ссылок. Ссылки в формальных параметрах функций.

### Ответ по Java:

1. **Ковариантность массивов (Слайд 60):** Массивы ковариантны. Если `S` — наследник `T`, то `S[]` — подтип `T[]`. Это небезопасно и может привести к `ArrayStoreException` в runtime.
2. **Инвариантность обобщённых классов (Слайд 61):** Обобщённые классы инвариантны. Если `S` — наследник `T`, то `G<S>` и `G<T>` не связаны. Это безопасно.
3. **Шаблоны (Wildcards) (Слайд 62):** `Stack<?>` — неявный супертип для любого `Stack`.
4. **Особенности шаблонов (Слайд 63):** Можно объявить переменную типа `Stack<?>`, но нельзя создать объект `new Stack<?>()`. Переменной `Stack<?>` можно присвоить любой `Stack<T>`.

### Ответ по C++:

1. **Объявление переменных ссылочного типа (Слайд 7)**
  - **Ссылка** – это типизированный указатель, который не может быть нулевым и к которому неприменимы арифметические операции.
  - Для объявления используется префиксный декларатор `&`.
  - **Синтаксис:** `тип& имя_переменной;`
2. **Инициализация и использование ссылок (Слайды 7-8)**
  - Ссылка **должна быть обязательно инициализирована** при объявлении.
  - После инициализации ссылка не может быть изменена, чтобы указывать на другой объект.
  - Для доступа к значению не требуется операция разыменования `*`.

```
int x = 10;
int& y = x; // y указывает на x
y = 20;     // теперь x равен 20
```

3. **Ссылки в формальных параметрах функций (Слайд 9)**
  - Использование ссылок в качестве параметров позволяет имитировать передачу параметров по ссылке (как `var`-параметры в Pascal).
  - Это позволяет функции изменять значения переменных, переданных ей при вызове.

```
void swap(int& a, int& b) { /* ... */ }
```

---

## Вопрос 16

- **Вопрос по Java:** Полиморфизм на основе шаблонов. Частичные шаблоны. Шаблоны, ограниченные сверху. Методы, недоступные у шаблонов, ограниченных сверху.

- **Вопрос по C++:** Проблема копирования объектов. Объявление конструктора копий. Перегруженная операция присваивания.

#### Ответ по Java:

1. **Полиморфизм на основе шаблонов (Слайд 64):** Шаблоны (?) позволяют писать методы, работающие с обобщенными классами с любым фактическим типом.
2. **Частичные шаблоны (Слайд 65):** Если у класса несколько типовых параметров, можно использовать ? для части из них. `Map<Integer, ?> intMap;`
3. **Шаблоны, ограниченные сверху (Слайд 66):** ? `extends X`. Означает "любой тип, который является X или его наследником". Используется для чтения (Producer).
4. **Методы, недоступные у шаблонов, ограниченных сверху (Слайд 67):** Нельзя вызывать методы, принимающие параметр типа ? (т.е. нельзя записывать в структуру), так как точный тип неизвестен.

#### Ответ по C++:

Этот вопрос дублирует вопрос №7 по C++. Ответ см. выше.

1. **Проблема копирования объектов (Слайд 32)**
  - По умолчанию C++ выполняет **побитовое копирование** внутреннего состояния объекта.
  - Это становится проблемой для классов, содержащих указатели на динамически выделенную память. В результате копирования два объекта (оригинал и копия) будут указывать на одну и ту же область памяти. Уничтожение одного объекта приведет к освобождению этой памяти, и указатель во втором объекте станет "висячим".
2. **Объявление конструктора копий (Слайд 33)**
  - Для создания "правильных" (глубоких) копий необходимо объявить **конструктор копий**. Он вызывается при инициализации объекта другим объектом.
  - **Прототип:** `Имя_класса(const Имя_класса& obj);`
  - В его теле нужно выделить новую память и скопировать в нее данные из исходного объекта.
3. **Перегруженная операция присваивания (Слайд 34)**
  - Проблема копирования возникает и при присваивании. Для ее решения перегружается оператор `=`.
  - **Прототип:** `Имя_класса& operator=(const Имя_класса& obj);`
  - Реализация должна включать **проверку на самоприсваивание** (`if (this != &obj)`), освобождение старой памяти, выделение новой и копирование данных.

---

#### Вопрос 17

- **Вопрос по Java:** Шаблоны, ограниченные снизу. Методы, недоступные у шаблонов, ограниченных снизу.
- **Вопрос по C++:** Ссылки как возвращаемые значения функций. Константные ссылки.

#### Ответ по Java:

1. **Шаблоны, ограниченные снизу (Слайд 68):** ? `super X`. Означает "любой тип, который является X или его суперклассом". Используется для записи (Consumer).

2. **Методы, недоступные у шаблонов, ограниченных снизу (Слайд 69):** Нельзя вызывать методы, возвращающие значение типа `?` (т.е. нельзя читать из структуры, кроме как в `Object`), так как точный тип неизвестен.

**Ответ по C++:**

Этот вопрос дублирует вопрос №14 по C++. Ответ см. выше.

1. **Ссылки как возвращаемые значения функций (Слайд 10)**

- Функция может возвращать ссылку. Это позволяет использовать результат вызова функции в левой части операции присваивания, то есть изменять значение, на которое указывает возвращенная ссылка.

```
char& ith(char *s, int i) { return s[i]; }  
// ...  
ith(s, 2) = 'x'; // Изменяем 2-й символ в строке s
```

- Важно:** нельзя возвращать ссылку на локальную переменную, так как она будет уничтожена после выхода из функции.

2. **Константные ссылки (Слайд 11)**

- Объявляются с модификатором `const`.
- Синтаксис:** `const тип& имя_переменной;`
- Константная ссылка позволяет читать значение, на которое она указывает, но **запрещает его изменять**. Попытка изменить значение через константную ссылку приведет к ошибке компиляции.

```
const int& y = x;  
y = 20; // Ошибка!
```

---

## Вопрос 18

- Вопрос по Java:** Типовые параметры у методов. Объявление и вызов обобщённого метода.
- Вопрос по C++:** Объявление класса. Секции в объявлении класса.

**Ответ по Java:**

1. **Типовые параметры у методов (Слайд 71):** Методы (статические и экземплярные) могут иметь свои собственные типовые параметры, что делает их обобщёнными.
2. **Объявление и вызов (Слайд 72):**
- Объявление:** Типовые параметры указываются в `<T>` **перед** типом возвращаемого значения. `public static <T> void someMethod(T arg) { ... }`
  - Вызов:** `Класс.<Тип>метод(...)`. Чаще всего компилятор сам выводит тип, и явное указание `<Тип>` не требуется.

**Ответ по C++:**

Этот вопрос дублирует вопрос №13 по C++. Ответ см. выше.

### 1. Объявление класса (Слайд 12)

- Классы объявляются с помощью конструкции `class Имя { ... };`.
- В отличие от `struct`, где члены по умолчанию `public`, в `class` члены по умолчанию `private` (недоступны извне).
- В теле класса располагаются объявления полей и прототипы методов.

### 2. Секции в объявлении класса (Слайд 13)

- Для управления доступом к членам класса используются секции-спецификаторы:
  - `public::`: члены видны отовсюду.
  - `private::`: члены видны только из методов данного класса.
  - `protected::`: члены видны из методов данного класса и его классов-наследников.
- Секции могут идти в любом порядке и количестве.

---

## Вопрос 19

- **Вопрос по Java:** Нештатные ситуации. Перехват нештатных ситуаций.
- **Вопрос по C++:** Объявление переменных ссылочного типа. Инициализация и использование ссылок. Ссылки в формальных параметрах функций.

### Ответ по Java:

1. **Нештатные ситуации (Слайд 73):** Ситуации, когда выполнение кода невозможно. Делятся на **исключительные** (внешние причины, например, ошибка файла) и **ошибочные** (ошибки в коде, например, деление на ноль).
2. **Перехват нештатных ситуаций (Слайд 74):** Механизм для продолжения работы программы.  
Два способа:
  - **Обработка кодов возврата:** проверка значения, возвращаемого функцией.
  - **Обработка исключений:** генерация объекта-исключения и передача управления специальному обработчику.

### Ответ по C++:

Этот вопрос дублирует вопрос №15 по C++. Ответ см. выше.

### 1. Объявление переменных ссылочного типа (Слайд 7)

- **Ссылка** – это типизированный указатель, который не может быть нулевым и к которому неприменимы арифметические операции.
- Для объявления используется префиксный декларатор `&`.
- **Синтаксис:** `тип& имя_переменной;`

### 2. Инициализация и использование ссылок (Слайды 7-8)

- Ссылка **должна быть обязательно инициализирована** при объявлении.
- После инициализации ссылка не может быть изменена, чтобы указывать на другой объект.
- Для доступа к значению не требуется операция разыменования `*`.

```
int x = 10;
int& y = x; // y указывает на x
y = 20;     // теперь x равен 20
```

### 3. Ссылки в формальных параметрах функций (Слайд 9)

- Использование ссылок в качестве параметров позволяет имитировать передачу параметров по ссылке (как `var`-параметры в Pascal).
- Это позволяет функции изменять значения переменных, переданных ей при вызове.

```
void swap(int& a, int& b) { /* ... */ }
```

---

## Вопрос 20

- **Вопрос по Java:** Иерархия классов исключений. Методы класса `Throwable`.
- **Вопрос по C++:** Объявление полей. Определения статических полей. Объявление методов. Виртуальные и абстрактные методы.

### Ответ по Java:

#### 1. Иерархия классов исключений (Слайд 75):

- `Throwable`
  - `Error`: фатальные системные ошибки (`StackOverflowError`).
  - `Exception`: обычные исключения.
    - `RuntimeException`: ошибки в коде, непроверяемые (`NullPointerException`).
    - Остальные наследники `Exception`: проверяемые исключения (`IOException`).

#### 2. Методы класса `Throwable` (Слайд 76):

- `String getMessage()`: сообщение об ошибке.
- `Throwable getCause()`: вложенное исключение-причина.
- `void printStackTrace()`: вывод стека вызовов.

### Ответ по C++:

Этот вопрос дублирует вопрос №12 по C++. Ответ см. выше.

#### 1. Объявление полей (Слайд 15)

- Экземплярные поля объявляются так же, как поля структур в C.
- Статические поля объявляются с модификатором `static`.

#### 2. Определения статических полей (Слайд 16)

- Каждое статическое поле должно быть **определено** (инициализировано) в одном из `src`-файлов проекта, так как оно является, по сути, глобальной переменной.
- Синтаксис: `тип Имя_класса::имя_поля = значение;`

#### 3. Объявление методов (Слайд 17)

- Объявление состоит из **прототипа** в теле класса и **определения** в `src`-файле.
- Прототипы статических методов объявляются с `static`.

#### 4. Виртуальные и абстрактные методы (Слайд 18)

- **Виртуальный метод:** для включения позднего связывания прототип метода помечается модификатором `virtual`. `virtual double dist();`

- **Абстрактный (чисто виртуальный) метод:** прототип такого метода заканчивается на `= 0;`. Класс, содержащий хотя бы один абстрактный метод, автоматически становится абстрактным. `virtual double dist() = 0;`
- 

## Вопрос 21

- **Вопрос по Java:** Неперехваченные исключения. Операторы перехвата исключений.
- **Вопрос по C++:** Создание объектов в динамической памяти. Создание массивов в динамической памяти. Удаление объектов и массивов.

### Ответ по Java:

1. **Неперехваченные исключения (Слайд 78):** Приводят к аварийному завершению программы с выдачей дампа стека вызовов.
2. **Операторы перехвата исключений (Слайд 79):**
  - **try:** блок с кодом, который может породить исключение.
  - **catch:** блок-обработчик для исключения определенного типа.
  - **finally:** блок, который выполняется всегда, независимо от того, было исключение или нет (для освобождения ресурсов).

### Ответ по C++:

Этот вопрос дублирует вопрос №10 по C++. Ответ см. выше.

1. **Создание объектов в динамической памяти (Слайд 21)**
    - Динамическое выделение памяти под объект выполняется операцией `new`. Она совмещает выделение памяти и вызов конструктора.
    - **Синтаксис:** `new имя_класса(фактические_параметры_конструктора);`
    - Операция возвращает указатель на созданный объект. `Point *p = new Point(1.5, 2.3);`
  2. **Создание массивов в динамической памяти (Слайд 22)**
    - Для создания массивов используется специальная форма `new`.
    - **Синтаксис:** `new тип_элемента[размер];`
    - **Важно:** для создания массива объектов в динамической памяти у класса должен быть определён **конструктор по умолчанию**, который будет вызван для каждого элемента массива.
  3. **Удаление объектов и массивов (Слайд 23)**
    - Так как в C++ нет сборщика мусора, все объекты, созданные через `new`, должны быть явно освобождены оператором `delete`.
    - **Для одиночного объекта:** `delete указатель_на_объект;`
    - **Для массива:** `delete [] указатель_на_массив;` (использование `[]` обязательно для корректного вызова деструкторов для всех элементов).
- 

## Вопрос 22

- **Вопрос по Java:** Порождение исключения.
- **Вопрос по C++:** Объявление конструкторов.

**Ответ по Java:****1. Порождение исключения (Слайд 81):**

- Выполняется оператором `throw` **исключение**;
- Если исключение **проверяемое** (не наследник `Error` или `RuntimeException`), то метод, который его порождает, должен либо перехватить его в `try-catch`, либо объявить в своей сигнатуре с помощью `throws`.
- `void myMethod() throws MyCheckedException { ... }`

**Ответ по C++:**

Этот вопрос дублирует вопрос №11 по C++. Ответ см. выше.

**1. Объявление конструкторов (Слайд 19)**

- Объявление конструктора состоит из **прототипа** в теле класса и **определения** в `src`-файле.
- Имя конструктора **совпадает с именем класса**.
- Тип возвращаемого значения **не указывается**.
- Чтобы запретить создание экземпляров, конструктор можно объявить в `private` или `protected` секции.

**2. Особенности (Слайды 19-20)**

- В C++ **никогда не следует вызывать виртуальные методы из конструктора**, так как это может привести к проблемам при наследовании и переопределении.
- Для доступа к полям объекта из конструктора используется указатель `this`. `this->x = x;`

---

**Вопрос 23**

- **Вопрос по Java:** Понятие функтора. Реализация метода `map` в функторе `Roots`. Перебор корней в функторе `Roots`. Использование функтора `Roots`.
- **Вопрос по C++:** Объявление деструктора. Объекты в автоматической памяти. Автоматический вызов деструктора.

**Ответ по Java:**

- Понятие функтора (Слайд 83):** Обобщенный класс-контейнер `Functor<T>`, объекты которого неизменяемы и у которого есть метод `map`. Метод `map` принимает функцию `T -> R` и возвращает новый `Functor<R>`.
- Реализация `map` и `forEach` (Слайды 86-87):** В примере `Roots` метод `map` применяет функцию к каждому корню и возвращает новый `Roots`. Метод `forEach` применяет `Consumer` для перебора корней.
- Использование (Слайды 88-89):** Позволяет выстраивать декларативные цепочки преобразований данных: `Roots.of(...).map(...).map(...).forEach(...)`.

**Ответ по C++:**

Этот вопрос дублирует вопрос №9 по C++. Ответ см. выше.

**1. Объявление деструктора (Слайд 24)**

- **Деструктор** – это экземплярный метод, предназначенный для освобождения ресурсов, принадлежащих объекту, непосредственно перед его уничтожением.
- В C++ без сборки мусора деструкторы жизненно необходимы.
- **Прототип:** `~имя_класса();`
- Деструктор вызывается при уничтожении объекта, в частности, его автоматически вызывает оператор `delete`.

## 2. Объекты в автоматической памяти (Слайд 26)

- В C++, в отличие от Java, объекты могут размещаться в автоматической памяти (на стеке) как локальные переменные.
- Синтаксис: `Имя_класса имя_переменной(параметры_конструктора);`

## 3. Автоматический вызов деструктора (Слайд 26, 28)

- При выходе из блока (например, из функции), где была объявлена переменная с объектом в автоматической памяти, для этого объекта **автоматически вызывается деструктор**. Это обеспечивает детерминированное управление ресурсами (принцип RAII).

## Вопрос 24

- **Вопрос по Java:** Введение в монады на примере класса `Roots`. Добавление метода `flatMap` в класс `Roots`. Использование метода `flatMap`.
- **Вопрос по C++:** Объекты в глобальной памяти. Объекты в полях других объектов.

### Ответ по Java:

1. **Введение в монады (Слайд 90):** Монада решает проблему, когда функция, переданная в `map`, сама возвращает функтор (например, `Roots<Roots<Double>>`).
2. **Метод `flatMap` (Слайд 91):** Принимает функцию `T -> Monad<R>` и "уплощает" результат, объединяя вложенные монады в одну `Monad<R>`.
3. **Использование (Слайд 92):** Позволяет элегантно решать задачи с вложенными вычислениями, сохраняя "плоскую" структуру результата. `Roots.of(...).flatMap(...)`.

### Ответ по C++:

Этот вопрос дублирует вопрос №8 по C++. Ответ см. выше.

## 1. Объекты в глобальной памяти (Слайд 29)

- Если объект размещается в глобальной переменной (или статическом поле класса), его конструктор вызывается **до** передачи управления в функцию `main`.
- Деструктор для такого объекта вызывается **после** завершения функции `main`.

```
Demo d(100); // Глобальный объект
int main() { /* ... */ }
// Вывод: cons:100 main destr:100
```

## 2. Объекты в полях других объектов (Слайды 30-31)

- Если объект класса `X` является полем `x` объекта класса `Y`, то конструктор для поля `x` вызывается из конструктора `Y` через **список инициализации**. `Y::Y(...) : x(...) { ... }`

- Деструктор для объекта-поля (`d` в классе `Cont`) вызывается **автоматически** при уничтожении объекта-контейнера (`c`).

---

## Вопрос 25

- **Вопрос по Java:** Определение монады. Проблема нулевых ссылок. «Заворачивание» значения в `Optional`. «Разворачивание» `Optional`. Композиция частичных функций.
- **Вопрос по C++:** Основной способ наследования в C++. Переопределение методов.

### Ответ по Java:

1. **Определение монады (Слайд 93):** Функтор с дополнительным ассоциативным методом `flatMap`.
2. **Проблема нулевых ссылок (Слайд 94):** Использование `null` для обозначения отсутствия значения приводит к необходимости постоянных проверок и риску `NullPointerException`.
3. **`Optional<T>` (Слайды 95-98):** Монада, решающая проблему `null`.
  - **Заворачивание:** `Optional.ofNullable(value)`.
  - **Разворачивание (безопасное):** `ifPresent(consumer)`.
4. **Композиция частичных функций (Слайд 99):** Функции, которые могут вернуть `null`. Их композиция без монад требует каскада `if`-проверок.

### Ответ по C++:

Этот вопрос дублирует вопрос №6 по C++. Ответ см. выше.

1. **Основной способ наследования в C++ (Слайд 35)**
  - Синтаксис одиночного наследования: `class ИмяПроизводного: public ИмяБазового { ... };`
  - В конструкторе производного класса необходимо явно вызывать конструктор базового класса (если у него нет конструктора по умолчанию) через **список инициализации**.  
`Dog::Dog(...) : Animal(...) { ... }`
2. **Переопределение методов (Слайды 37-38)**
  - Для переопределения метода, унаследованного от базового класса, достаточно объявить и определить этот метод в производном классе.
  - Чтобы переопределение работало полиморфно (через указатель на базовый класс), метод в базовом классе должен быть объявлен как **`virtual`**.

```
// В базовом классе
virtual void draw();
// В производном классе
virtual void draw(); // Переопределяет метод базового класса
```

---

## Вопрос 26

- **Вопрос по Java:** Метод `map`. Композиция частичных функций с использованием `Optional`.
- **Вопрос по C++:** Виртуальное наследование.

**Ответ по Java:**

1. **Метод `map` в `Optional` (Слайд 101):** Принимает функцию-преобразователь. Если `Optional` не пуст, применяет функцию к его значению и заворачивает результат в новый `Optional` (через `ofNullable`). Если `Optional` пуст, возвращает пустой `Optional`.
2. **Композиция с `Optional` (Слайд 102):** Позволяет выстраивать цепочки вызовов `map` для композиции частичных функций, избавляя от явных проверок на `null`.  
`Optional.ofNullable(x).map(f).map(g).`

**Ответ по C++:**

Этот вопрос дублирует вопрос №2 по C++. Ответ см. выше.

1. **Проблема, решаемая виртуальным наследованием (Слайды 46-47)**
  - При множественном наследовании может возникнуть "ромбовидная" (противоречивая) иерархия, когда класс-наследник получает один и тот же базовый класс по нескольким путям.
  - Основная проблема такой иерархии — **многократное включение полей базового класса** в производный класс. Например, если `D` наследует `B` и `C`, а `B` и `C` наследуют `A`, то объект класса `D` будет содержать поля из `A` дважды.
2. **Виртуальное наследование (Слайд 48)**
  - Это способ реализации наследования, который **гарантирует, что базовый класс не будет включён в производный класс более чем в одном экземпляре**, даже если он встречается в иерархии несколько раз.
  - **Синтаксис:** При наследовании используется ключевое слово `virtual`.  
`class B: public virtual A { ... }; class C: public virtual A { ... }; class D: public B, public C { ... }; // Поля из A будут включены только один раз`

---

**Вопрос 27**

- **Вопрос по Java:** Потоки как развитие идеи итераторов. Преобразование последовательностей. `Filter` и `map`, `flatMap`. Собирающие последовательности.
- **Вопрос по C++:** Наследование от нескольких базовых классов. Противоречия в именах наследуемых членов класса. Разрешение противоречий в именах.

**Ответ по Java:**

1. **Потоки (`Stream<T>`) (Слайд 103):** Развитие идеи итераторов, представляет последовательность значений. Получается из коллекций методом `stream()`.
2. **Преобразование (Слайд 104):**
  - `filter(predicate)`: отбирает элементы.
  - `map(mapper)`: преобразует каждый элемент.
  - `flatMap(mapper)`: преобразует каждый элемент в поток и "уплощает" результаты в один поток.
3. **Собирающие (Слайд 107):** Потребление значений из потока и запись в контейнер с помощью коллекторов и метода `collect()`.

**Ответ по C++:**

Этот вопрос дублирует вопрос №4 по C++. Ответ см. выше.

### 1. Наследование от нескольких базовых классов (Слайд 43)

- C++ поддерживает множественное наследование.
- **Синтаксис:** Базовые классы перечисляются через запятую после двоеточия. `class Y: public X1, public X2, ..., public Xn { ... };`
- В конструкторе производного класса должны быть вызваны конструкторы всех базовых классов (кроме тех, у кого есть конструктор по умолчанию).

### 2. Противоречия в именах наследуемых членов (Слайд 44)

- При множественном наследовании может возникнуть **неоднозначность (ambiguity)**, если в производный класс из разных базовых классов приходят члены (методы или поля) с одинаковыми именами и сигнатурами.
- Попытка обратиться к такому члену без уточнения приведет к ошибке компиляции. `c.m();`  
`// error: request for member 'm' is ambiguous`

### 3. Разрешение противоречий в именах (Слайд 45)

- Для разрешения неоднозначности необходимо использовать **квалифицированные имена**, явно указывая, из какого базового класса вызывается член. `c.A::m();` // ОК, вызывается метод m из класса A

---

## Вопрос 28

- **Вопрос по Java:** Механизм пакетов. Создание пакета. Правила видимости для классов пакета и их членов.
- **Вопрос по C++:** Иерархия наследования и классы противоречия. Основная проблема противоречивых иерархий.

### Ответ по Java:

1. **Механизм пакетов (Слайд 108):** Контейнер классов, предоставляющий отдельное пространство имён и управление доступом. Служит для структурирования больших систем.
2. **Создание пакета (Слайд 110):** В начало файла добавляется директива `package имя_пакета;`. Файлы пакета должны лежать в каталоге, путь к которому соответствует имени пакета.
3. **Правила видимости (Слайд 111):**
  - **public** классы видны отовсюду.
  - **Непубличные классы** (без модификатора) видны только внутри своего пакета.
  - **Члены без модификатора** (package-private) видны только внутри своего пакета.

### Ответ по C++:

Этот вопрос дублирует вопрос №3 по C++. Ответ см. выше.

### 1. Иерархия наследования и классы противоречия (Слайд 46)

- **Иерархия наследования** – это ориентированный ациклический граф, узлами которого являются классы.
- Класс B называется **классом противоречия**, если существует такой класс A, что в иерархии можно провести не менее двух непересекающихся путей из A в B. Иерархия с такими классами называется **противоречивой**.

## 2. Основная проблема противоречивых иерархий (Слайд 47)

- Основной проблемой является **возможность многократного включения полей базового класса** в производный класс.
- Пример:**

```
class A { private: int x; };  
class B: public A {};  
class C: public A {};  
class D: public B, public C {};
```

- В этом примере поле **x** будет содержаться в объекте класса **D** дважды.

---

## Вопрос 29

- Вопрос по Java:** Импорт классов из другого пакета. Импорт статических членов класса.
- Вопрос по C++:** Динамическое приведение типов.

### Ответ по Java:

#### 1. Импорт классов (Слайд 112):

- По полному имени:** `java.util.HashSet`.
- Импорт класса:** `import java.util.HashSet;`
- Импорт пакета:** `import java.util.*;`

#### 2. Импорт статических членов (Слайд 113):

- Позволяет обращаться к статическим членам без имени класса.
- Синтаксис:** `import static java.lang.Math.*;`
- Пример:** `cos(PI / 3);` вместо `Math.cos(Math.PI / 3);`

### Ответ по C++:

Этот вопрос дублирует вопрос №5 по C++. Ответ см. выше.

#### 1. Операция **dynamic\_cast** (Слайд 39)

- Выполняет динамическое (во время выполнения) приведение типа объекта.
- Синтаксис:** `dynamic_cast<T>(obj)`
- Поведение:**
  - Проверяет, является ли тип **T** одним из типов объекта **obj**.
  - При работе с **указателями**: если приведение успешно, возвращает указатель на объект, иначе возвращает **нулевой указатель**.
  - При работе со **ссылками**: если приведение успешно, возвращает ссылку, иначе порождает исключение `std::bad_cast`.

#### 2. Ограничение на использование (Слайд 40)

- dynamic\_cast** можно применять только к **полиморфным типам**, то есть к классам, в которых есть хотя бы один **виртуальный метод**. Если тип не полиморфный, возникнет ошибка компиляции. Это связано с тем, что информация о типе, необходимая для приведения, хранится в объекте только для полиморфных классов.